Additional Notes

WQD7003: Week 4's Lecture

1 Normalization

Example Output

Method	Feature_A	Feature_B	Feature_C
Original	10–50	100–10000	0.1–1.0
Min–Max	0.0–1.0	0.0–1.0	0.0–1.0
Z-Score	-1.41–1.41	-0.61–1.53	-1.29–1.12
Decimal Scaling	0.01–0.05	0.01–1.00	0.01–0.10

Summary of Use Cases

Method	Best For	Examples
Min–Max	Bounded ranges, preserving relationships	Neural networks, KNN, K-Means
Z-Score	Centered and scaled data	Linear/Logistic regression, PCA
Decimal Scaling	Simple scaling, no statistics needed	Quick normalization, data prep

Normalization	Real-World Domain	Example Application	Why It's Used
Min–Max	Computer Vision	Normalizing pixel intensities (0–255 → 0–1)	Keeps data in bounded range for neural nets
Min–Max	Finance	Scaling stock prices for clustering	Prevents large-value features from dominating
Z-Score	Banking	Credit scoring models	Equalizes influence of features with different units
Z-Score	Healthcare	Comparing lab test results	Enables fair comparison of varied metrics
Z-Score	Recommender Systems	User rating normalization	Removes personal bias in rating scales
Decimal Scaling	IoT Devices	Scaling sensor readings on low-power chips	Lightweight, integer-based normalization
Decimal Scaling	Telecom	Reducing signal magnitude for transmission	Keeps values within manageable numeric range

2 Data Reduction Strategy

Concept	Purpose	Real Example
Data Reduction Strategy	Reduce dataset size while preserving useful information	Summarizing a 10TB retail dataset into monthly sales reports
Data Cube Aggregation	Aggregate data at higher levels to minimize size	Total calls per region in a telecom database
Attribute Subset Selection	Keep only the most relevant variables	Choosing "age," "income," and "credit score" for a loan approval model

3 Data Reduction Strategy: Feature Selection

1) Attribute (Feature) Subset Selection

Idea. We want the *smallest* set of original features that still makes the classes easy to separate. Exhaustively trying every subset is impossible in practice because there are 2^d subsets.

Why do it?

- Faster training/inference
- Less overfitting, easier to interpret
- Lower data collection cost

Mini-example (spam filter).
Suppose you extracted 20 features from emails (e.g., "contains 'free'", "num links", "sender domain", "all caps ratio"…). If a classifier trained on just 4 of them—say {"num links", "sender domain", "all caps ratio", "has attachment"}—gets the same accuracy as using all 20, then those 4 are your selected subset.

Common search heuristics (since 2^d is huge):

- **Forward selection**: start empty, add the best next feature at each step.
- **Backward elimination**: start with all features, remove the worst one each step.
- **Stepwise (both ways)**: at each step you may add or remove a feature based on the current score.
- **Embedded methods**: the learner chooses features while training (e.g., decision trees).

2) Example of Decision-Tree Induction → Feature Selection for Free

The slide shows features $\{A1, A2, A3, A4, A5, A6\}$. A learned tree splits on **A4** at the root, then on **A1** and **A6**. Only these three are used anywhere in the tree, so the *effective* feature set is $\{A1, A4, A6\}$.

How the tree chose them.
At each node, the algorithm picks the feature that most reduces impurity (e.g., highest **information gain** (entropy) or **Gini** decrease). Features that never get picked were not helpful given those already used.

Mini-example (loan default).

- $A4$: debt-to-income ratio (first split)
- $A1$: has mortgage? (left branch)

- $A6$: number of late payments (right branch)
  Even if you engineered 50 features, the trained tree might only ever use 3–6. You can drop the rest for that model.

3) Heuristic Feature Selection Methods (what "best" means)

- **Best single features (filters).** Rank each feature by a univariate criterion—e.g., mutual information with the label, chi-square for categorical, ANOVA F-test for numeric—then keep the top k.
  - *Example:* For a disease classifier, "CRP level" and "age" have strongest univariate association → pick top 10.
- **Best stepwise selection (wrappers).** Use a model + cross-validation score as the objective while adding features one by one.
  - *Example:* Start with none; adding "CRP" lifts CV AUC from .50→.71; adding "age" to .78; adding "BMI" to .80; adding "cholesterol" barely helps (to .801), so stop.
- **Stepwise elimination (wrappers/filters).** Start with all features; iteratively remove the least helpful (or highest p-value) until performance drops.
  - *Example:* Fit a linear/logistic model; drop the highest p-value term each round until all remaining are < 0.05.
- **Combined selection & elimination.** After each addition, check whether any currently-kept feature became redundant; remove it.
- **Branch-and-bound (optimal under assumptions).** Systematically prunes the search tree using bounds on the best possible score; works when the objective is monotonic and you can compute tight bounds. Less common in day-to-day ML.

4) Dimensionality Reduction via PCA (not selection; transformation)

Idea. Replace your $d$original features by k**new, orthogonal** features (principal components) that capture as much variance as possible. You *transform* the space instead of choosing a subset.

Steps (practical view).

1. **Normalize/standardize** features (zero mean, unit variance).
2. Compute the **covariance matrix**; find its eigenvectors/eigenvalues.
3. Sort eigenvectors by eigenvalues (variance explained); keep the top k.
4. Project your data onto these $k$vectors to get PC1..PCk.

Mini-example (height vs. arm-span).
You have 2 features that are highly correlated. PCA finds:

- **PC1** ≈ (height + arm-span)/√2 → explains ~95% of variance.
- **PC2** ≈ (height − arm-span)/√2 → small remaining variance.
  You can keep only PC1, shrinking 2D → 1D with little information loss.

Another example (images).
MNIST digits: 784 pixels → project to 50 PCs and keep ~90–95% variance; training a classifier becomes much faster with little accuracy loss.

When PCA fits / doesn't:

- ✓ Numeric features, many dimensions, strong correlations, you're okay with **less interpretability**.
- ✕ When interpretability matters (you want the original feature names) or features are categorical (unless you first embed/ordinal-encode).
- **Putting it together: feature selection vs. PCA**

Goal	Method	Pros	Cons
Keep a subset of original features	Forward/backward/stepwise, filters, trees, L1	Interpretable; cheaper data collection	Greedy searches may miss best subset; can ignore feature interactions (filters)
Create new compact features	PCA (or other DR: ICA, t-SNE/UMAP for viz)	Captures variance; de-correlates; speeds up	Components are linear combos → harder to explain; numeric-only assumption